

Machine Learning Techniques for Movie Recommendation: A Matrix Factorization Perspective

Ashish Kumar Nayyar*

Abstract

Recommender Systems have become a cornerstone of modern digital platforms; they are providing personalized content and suggestions thereby significantly enhancing overall user experience. This paper explores the real-world applications of recommender systems with a particular focus on the movie domain. The paper discusses various areas where such recommender systems can efficiently be deployed and used, highlighting their relevance in entertainment, e-commerce, and content delivery platforms. Movie recommendation systems, in particular, leverage machine-learning techniques to analyze users' past behavior, preferences, and viewing patterns to generate tailored suggestions. These systems improve user satisfaction and play an important role in driving user engagement and platform retention. Moreover, the integration of contextual information—such as time, location, and user mood—further refines the recommendation process, making it more dynamic and responsive. This paper presents a comprehensive overview of different approaches of machine learning being used in building movie recommendation systems, including collaborative filtering, content-based filtering, and hybrid models. The associated challenges with data sparsity, scalability, and cold-start problems are discussed, and propose potential solutions to address them. This paper aims to contribute to the growing body of knowledge in personalized recommendation by offering insights into both the technical foundations and practical implications of movie recommendation systems in today's data-driven environment.

Keywords: *Matrix Factorization, Recommendation Systems, Movie Recommendation Systems, Content-Based Filtering and Collaborative Filtering*

* Assistant Professor, Department of Computer Science, Institute of Information Technology & Management, New Delhi

1. INTRODUCTION

In today's time when there is too much information available everywhere, people need an easy way of finding content that is useful or interesting to them. Movie recommendation systems along with other applications of recommender systems have gained significant attention. This is due to the vast and continually growing volume of digital entertainment content (Nilashi, M. et.al., 2018). These systems try to improve user experience by giving movie suggestions that match personal taste. They look at what a user likes, what they watched before, and other related factors. With the growth of e-commerce and digital platforms, there is now a huge amount of content available online. Because of this, users often find it difficult to choose what they actually want.

Recommender systems help solve this problem. They study user behavior and preferences. They also consider features of items or products. Based on this, they provide suggestions that are more relevant to the user. This paper focuses on the implementation of machine learning algorithms to build a movie recommendation system. The recommendation system gives an output of a handful of suggestions to the user by using an automatic filtering method that after taking certain inputs of the respective item.

Furthermore, it delivers as well as detects the information, which, for the user is likely to be useful, or interesting as using the above-mentioned technique, it narrows out the relevant suggestions that would be presented to the user after layers of filtering from the given information block. In the past few years, the amount of data created around the world has grown a lot, with more than 80% of it being produced during this period. This growth in content, along with the context surrounding it on the internet, has been notable, and the need for an application-like recommendation system has become an essential tool to have better-optimized results.

A recommendation system incorporates the user's needs into a user model for a series of individualized suggestions, then uses workable and Precise algorithms to align the user's preferences with the recommended products or services. Each recommender system involves three steps: gathering user preferences (input); computing the suggestion using suitable methods and techniques; and communicating the results to the users. The global positioning system is now used everywhere. It is even available on mobile phones through apps like Google Maps. People use it daily for directions and finding places. Recommendation systems are also very useful today. They look

at things like location, music taste, and shopping habits. Based on this, they suggest items that match the user's interest. However, many users still feel unsure about trusting these systems. This has been observed for a long time. In this work, we tried to build a movie recommendation system. It uses past movie ratings given by users. Then a machine-learning model predicts ratings for movies that the user has not watched yet. This helps in suggesting new movies. The system gives better results over time. It suggests movies similar to what the user already likes. It also shows the movie title along with its genres. This makes the output clear and easy to understand. Likewise, this beneficial product visualizes how the respective system has stood on its primal objectives, which can be formulated as follows:

A means of increasing sales, by encouraging users to make purchases using the desired recommender system, Drastically improving user experience, and Ultimately skyrocketing customer loyalty as the boom in experience, lends a helping hand in increasing the commitment of the users.

The remainder of the paper is followed in an organized manner: Section 2 states the historical description of recommendation system. Section 3 would be highlighting the various classifications of the recommender application along with the merits and demerits of each type. Section 4 describes the feedback techniques for more logical suggestions. Section 5 emphasizes the experimental setup of the work, consisting of project code snippets followed by their respective explanation. Experimental results are illustrated in Section 6 with conclusions mentioned in Section 7.

2. A RECOMMENDATION SYSTEM

The first and foremost recommendation system was built and introduced by Goldberg, Nichols, and Oki & Terry in the year of 1992. The main goal of a recommender system is to provide information to the user based on user's previous searches. It is true that not everyone has the same tastes, especially when looking at a large group of people. However, it has been clearly seen that even in such a challenging situation, there can still be patterns that connect groups of people, even if they have nothing else in common. Moreover, the recommendation system is an application that lends a helping hand to all ranks and files to have an extra quantity of context accompanied by the exquisite quality of content, every single of which is based on the previous searches, which all were inclined towards the interest of the respective user himself. Likewise, the Recommendation system

perceives the suggestion by taking account of all past taken course steps and running them from numerous techniques and algorithms, which, henceforth, provides the user with an appropriate recommendation in the current specified field of interest. Following Fig. 1 represents a hierarchical structure in total types of recommendation systems.

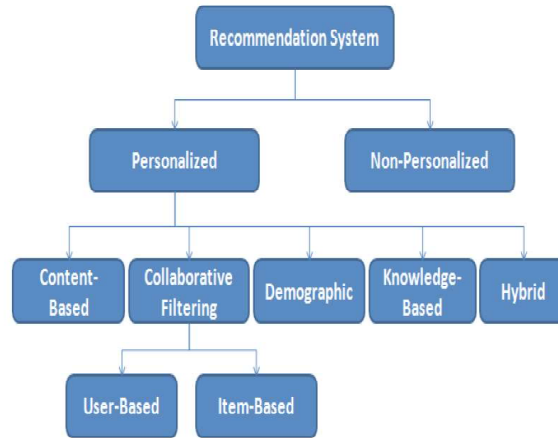


Fig.1: Types of Recommender Systems

3. CLASSIFICATIONS OF THE RECOMMENDER APPLICATION

By incorporating all three perspectives, such a model of the recommender system seeks to indicate users' preferred things according to their past interactions and organizational network interactions.

1. Interpersonal influence, which denotes whom you would trust;
2. Interest circle derivation, which denotes who shares your interests; and
3. User individual interest, which affects the goods you would be enthusiastic about.

Five different types of tailored recommendation systems are distinguished by their procedures for going about making recommendations.

Content-Based Filtering

This filtering is centered on the item's description and the consumer's preferences. Profoundly, the goal of these algorithms is to recommend goods or services that are alike to the services which users have previously liked or appreciated or have recently been looking at (Thorat, P.B., et.al. 2015) Figure.2 delineates a pictorial representation

of content-based filtering.

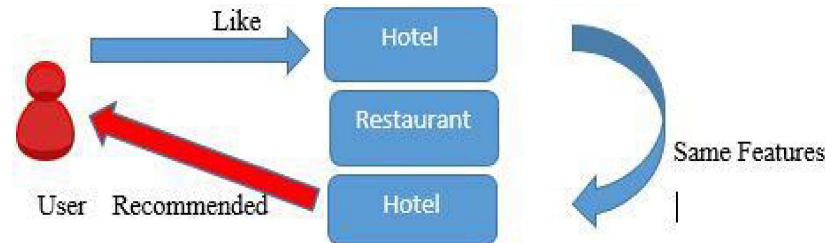


Fig.2: Representation of Content-Based Filtering

The merits are as follows:

- The data of other users is not necessary.
- There are no issues with data sparsity or cold start.

The demerits are as follows:

- Examination of the content is crucial in identifying the key characteristics of a product
- Evaluation of the quality of a product is challenging, and relying solely on the similarity calculation of product descriptions is insufficient.

Collaborative Filtering

Collaborative filtering technique involve building a system that takes into account the user's historical behavior, such as items previously rated, queried, or selected, and also incorporates similar choices made by other users. This data is then utilized to ascertain the item or rating which user may find appealing (Geetha, G. et.al. 2018) a clearer representation is shown in Figure 3.

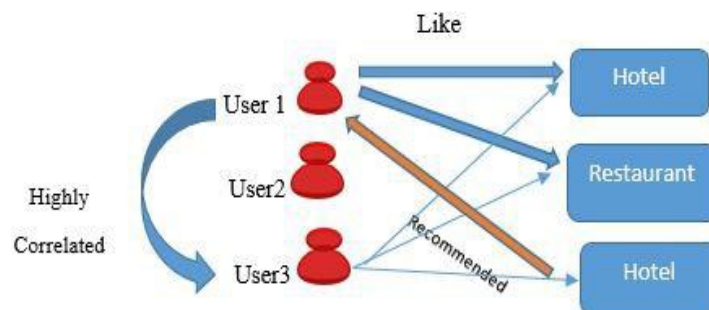


Fig.3: Representation of Collaborative Filtering

The merits are as follows:

- The excellence of the suggestion given can be estimated by the rating provided by the user.

The demerits are as follows:

- Cold start concerns with new items and distinct users.
- Stability vs. plasticity issue.

Demographic Filtering

The technique of demographic suggestion relies exclusively on the user's demographic data, such as age, gender, occupation, property ownership, and even location, to generate recommendations based on similarities in demographic characteristics (Liao, M. et.al. 2022).

The merits are as follows:

- Since the Item feature is not needed, the technique is domain independent.

The demerits are as follows:

- An issue of privacy mushrooms up due to the collection of demographic Information.
- Plasticity vs. stability issue.

Knowledge-Based Recommender System

This type of system gets deployed in some specific domains where the perch's history is petite. Insubstantially, the installed algorithm takes knowledge about the item into an account along with other aspects, such as the item's features, a user's preference, and a certain criterion of recommendation before giving out a suggestion (Uta, M. et al., 2024). There are two types of knowledge-based recommender systems.

1. Constraint-based
2. Case-based

Unlike constraint-based recommenders, which rely on a laid-out set of recommendation principles, case-based recommenders are focused on obtaining analogous items based on multiple kinds of resemblance metrics.

4. HYBRID RECOMMENDER SYSTEM

Hybrid recommender systems use by combining multiple

recommendation algorithms to create a stronger and more dependable foundation. By leveraging the strengths of different recommender systems and minimizing their limitations, we can build a more resilient and effective system. For example, collaborative filtering techniques may struggle with new products lacking ratings, while content-based methods rely on feature information about items. By combining these methods, we can recommend new products with greater precision and effectiveness (Bodduluri, K.C., 2024). In the bygone era, a few subsequent queries were understudied to construct a hybrid model, which is as follows:

1. Which techniques are required to be merged to achieve the desired business solution?
2. What is the suitable way to amalgamate multiple systems and their outputs to ensure accurate predictions?

5. FEEDBACK TECHNIQUES

Feedback is an important element of Recommender systems as it provides the necessary information for the system to make relevant suggestions based on the consumer's choices. Additionally, there are three distinct categories of feedback techniques (Dhar, S., 2024).

Implicit Feedback Technique (IFT)

In this approach, data is collected by observing how a user behaves while using a system. It does not depend on what the user knowingly shares. Instead, the system tracks actions in the background. Because of this, user interests and preferences are judged without direct permission. An IFT works by collecting and studying feedback using tools designed for a specific application. It depends on the type of system being used. This method is commonly seen in areas like mouse movement tracking, browsing history, and search activity analysis.

Explicit Feedback Technique (EFT)

This method is about asking users to rate a product using a score or number. Usually, people give ratings on a fixed scale, like out of 10. These ratings can then be analyzed to find averages, patterns, and trends. It helps in understanding how people feel about a product. EFT lets users clearly show their interest or preference for a particular item.

Hybrid Feedback Technique (HFT)

HFT can be formed by combining IFT and EFT, which utilizes

both numerical rating scores and social interactions to predict items that users may find appealing based on user's interests and choice.

6. EXPERIMENTAL SETUP

Under this section, various key aspects, namely the specific research methodology that was employed, the numerous tools, which were used, and certain open-source libraries that have been used, would be presented. The beauty of the designed Recommender system is its flexibility in nature and the compounding characteristic of being dynamic, as the key requirements of any system of using it are as basic as any other microcomputer, which is designated for a single user use, only embedded with a microprocessor as its CPU.

The system was built using a simple and practical approach. The dataset for this study was taken from Kaggle. The MovieLens (latest small) dataset was used. It includes user ratings for movies. It also has basic movie details. This data is useful for collaborative filtering. It also works well for matrix factorization methods. First, user-rating data was taken from a CSV file. This file had movie ratings given by different users. The data was then processed using Python and libraries like NumPy and Pandas. After that, matrix factorization was applied. In this step, the large user-item matrix was broken into two smaller matrices. This helped in handling sparse data and predicting missing ratings. Random values were assigned initially and then optimized using a cost function. The model kept improving by reducing the error between actual and predicted ratings. In the end, the system was able to suggest movies based on user preferences and past behavior. Likewise this methodology is shown in the following Fig.4 and heterogeneous modules are discussed briefly in the upcoming subsections.

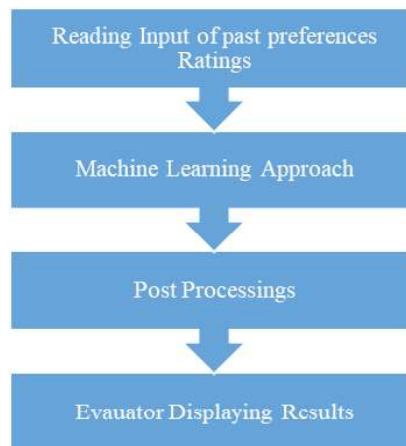


Fig.4: Application Architechure

Information Analysis

This sub-module consists of steps of reading the users' historical input from a CSV file, which consists of various ratings in numerical order of umpteen movies list.

Matrix Factorization

The basic idea behind Machine Learning techniques is to factorize the user-item rating matrix into two lower-dimensional matrices, one each for users and the items, by minimizing the reconstruction error. As the user item-rating matrix may be sparse, it is difficult to make accurate recommendations based on such matrix alone. Matrix factorization overcomes this limitation by reducing the dimensionality of the matrix, this helps in predicting ratings for items that a user has not rated yet. The prominent aspect of taking the data and breaking it down into smaller, easier-to-handle pieces makes it a suitable algorithm to be used for the Recommendation system. Matrix factorization is a powerful tool that can help businesses and users make better decisions by predicting what products they might enjoy based on their past preferences.

Tools Used

The programming language and open source libraries that are used briefly for developing and analyzing the system would be named as the tools used in building the respective system.

- **Python Programming Language:** Python version 3.9.12 on Windows 11 was used as is used for analyzing the text and text mining nowadays.
- **Numpy, Pandas, Pickle, and Scipy:** To regularly work with numerous arrays, create portable serialized representations of Python objects, and use functions for minimizing (or maximizing) objective functions, possibly subject to constraints, above mentions modules were used.

Snippets

The following Fig. 5 conspicuously represents a snippet on Python technique matrix factorization (Bobadilla, J., 2024) which tries to predict how much a user likes a product based on their past ratings. The code uses a method called matrix factorization, which breaks down the user-product rating matrix in 2 smaller matrices called P and Q.

The code first gets the shape of the rating matrix, which tells us

how many users and products there are. Then it checks if there are any missing ratings (represented by NaN) and creates a mask to mark those missing values.

Next, it replaces the missing ratings with 0 and creates two matrices P and Q filled with random numbers. These matrices are then combined into a single array and passed into a function called `fmin_cg` along with other arguments. `fmin_cg` is a function that tries to minimize the difference between the predicted and actual ratings using a cost function.

After `fmin_cg` has finished running, the new P and Q matrices are separated from the combined array and returned as `nP` and `nQ`. These matrices may be used to predict how much a user will like an unrated product.

```
def low_rank_matrix_factorization(ratings,mask,num_features,regularization_amount):
    num_users, num_products = ratings.shape
    if mask is None:
        mask = np.invert(np.isnan(ratings))
    ratings = np.nan_to_num(ratings)
    np.random.seed(0)
    P = np.random.randn(num_users, num_features)
    Q = np.random.randn(num_products, num_features)
    initial = np.append(P.ravel(), Q.ravel())
    args = (num_users, num_products, num_features, ratings, mask, regularization_amount)
    X = fmin_cg(cost, initial, fprime=gradient, args=args, maxiter=3000)
    nP = X[0:(num_users * num_features)].reshape(num_users, num_features)
    nQ = X[(num_users * num_features):].reshape(num_products, num_features)
    return nP, nQ
```

Fig.5: Matrix Factorisation

7. EXPERIMENTAL RESULTS

After having it run down through the matrix factorization algorithm, it displays certain recommendations of movies once received with a specific `user_id`. As every movie rating is per a unique user, an individual `user_id` has been assigned to the respective movie ratings made by it. Once a handful of suggestions is provided to the user, to get new recommendations, the system is equipped with a later program that gives a new start to the user by appending the previous new suggestions into the list of old ones. Once the whole optimization process has taken place and the past preferences list has been updated, it then provides the user with a new set of movie suggestions. A powerful technique, Matrix factorization widely used in recommender systems, can be enhanced with various methods, such as regularization, bias term, and ensemble learning, to improve its accuracy and robustness. Ultimately, one-step closer to making it more suitable and appropriate for the ever-changing dynamic surroundings.

Fig. 6. delineates the result after submitting a user_id to the system and consequently, the desired recommendation has been received consisting of Movie Title, its genre, and respective rating.

```

Enter a user_id to get recommendations (Between 1 and 100):
68
Movies we will recommend:

```

	movie_id	title	genre	rating
	12	Horrorfest	horror	4.831418
S20		Buy My App	comedy	4.822730
30		Post-Apocalyptia 2	sci-fi, thriller, mystery	4.811035
1		The Sheriff 1	crime drama, western	4.766537
25		Drugs & Guns	crime drama	4.704367

Fig.6: Movie Recommendations as an Output

8. CONCLUSION

We attempt to simply outline the many types of recommendation techniques and their types in this work. We also looked at different feedback strategies used in recommender systems. There is still a lot of scope for future work in this area. New strategies and features can be explored to make these systems better. If we combine recommender systems with machine learning and NLP, the results can improve a lot. Such systems can consider multiple factors at the same time. Matrix factorization is quite useful here. It helps in predicting what a user may like based on past choices. This can support both users and businesses in making better decisions. With machine learning, the system can learn from previous data. Over time, it improves its recommendations. It becomes smarter and can guess user preferences more accurately. Different methods are used to build these systems. These include collaborative filtering, content-based filtering, matrix factorization, deep learning, and hybrid approaches. Each method has its pros and cons. In most cases, combining methods gives better results. It leads to more personalized recommendations.

There are still some challenges. One is the cold-start problem. This happens when there is very little data for new users or items. Another issue is the exploration vs. exploitation trade-off. The system has to decide between trying new suggestions and sticking to safe ones. These problems need more research. This understanding can help researchers improve current systems. It can guide them in building better recommendation models. Overall, movie recommender systems make it easy for users to find new movies. They also help streaming

platforms keep users engaged. Even though there are some limitations, these systems are very important today. They play a big role in how people discover content.

REFERENCES

1. Nilashi, M. , Ibrahim, O. & Bagherifard, K. (2018). A recommender system based on collaborative filtering using ontology and dimensionality reduction techniques. *Expert Syst. Appl.*, 92, 507–520. <https://doi.org/10.1016/j.eswa.2017.09.058>.
2. Thorat, P.B., Goudar, R.M. & Barve, S. (2015). Survey on collaborative filtering, content-based filtering and hybrid recommendation system, *Int. J. Comput. Appl.*, 110(4).
3. Geetha, G., Safa, M. , Fancy, C. & Saranya, D. (2018). A hybrid approach using collaborative filtering and content based filtering for recommender system, *Journal of physics: Conference Series*, 12101.
4. Liao, M., Sundar, S.S. & Walther, J.B. (2022). User trust in recommendation systems: A comparison of content-based, collaborative and demographic filtering, In *Proceedings of the 2022 CHI conference on human factors in computing systems*, (pp. 1–14).
5. Uta, M. *et al.*, (2024). Knowledge-based recommender systems: overview and research directions, *Front. big Data*, 7, 1304439.
6. Bodduluri, K.C. , Palma, F. , Kurti, A. , Jusufi, I. & Löwenadler, H. (2024). Exploring the landscape of hybrid recommendation systems in e-commerce: A systematic literature review, *IEEE Access*, 12, 28273–28296.
7. Dhar, S. (2024). A Survey on domain of application of recommender system, In *How Mach. Learn. is Innov. Today's World A Concise Tech. Guid.*, (pp. 375–382).
8. Bobadilla, J., Dueñas-Lerin, J., Ortega, F. & Gutierrez, A. (2024). Comprehensive evaluation of matrix factorization models for collaborative filtering recommender systems, *arXiv Prepr. arXiv2410.17644*.